



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2223 — Teoría de Autómatas y Lenguajes Formales — 2' 2025
IIC2224 — Autómatas y Compiladores — 2' 2025

PAUTA EXAMEN

Pregunta 1

Sea L un lenguaje regular, y $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ un DFA tal que $\mathcal{L}(\mathcal{A}) = L$.

Sabemos que, si existe un autómata cuyo lenguaje sea $\text{Mitad}(L)$, entonces $\text{Mitad}(L)$ es regular.

Solución 1: Agregar estados iniciales y finales para saltarse u y w

Se define el NFA $\mathcal{A}^- = (Q, \Sigma, \Delta^-, I^-, F^-)$ a partir de $\mathcal{A}$, donde

- $\Delta^- = \{(q, a, \delta(q, a)) \mid q \in Q, a \in \Sigma\}$ (Mantener las transiciones iguales)
- $I^- = \{q \mid \exists u. \hat{\delta}(q_0, u) = q\}$ (Estados alcanzables desde q_0)
- $F^- = \{p \mid \exists w. \hat{\delta}(p, w) \in F\}$ (Estados de adonde se puede alcanzar un estado final)

P.D: $\text{Mitad}(L) = \mathcal{L}(\mathcal{A}^-)$.

($\subseteq$) Supongamos que $v \in \text{Mitad}(L)$. Sabemos que existen $u, w \in \Sigma^*$ tal que $u \cdot v \cdot w \in L$.

Por lo tanto, existen q_1, q_2 tal que $\hat{\delta}(q_0, u) = q_1$, $\hat{\delta}(q_1, v) = q_2$ y $\hat{\delta}(q_2, w) \in F$.

Por construcción de $\mathcal{A}^-$, sabemos que $q_1 \in I^-$, $q_2 \in F^-$ y que $(q_1, v) \vdash_{\mathcal{A}^-}^* (q_2, \epsilon)$.

Por lo tanto, $v \in \mathcal{L}(\mathcal{A}^-)$.

($\supseteq$) Supongamos que $v \in \mathcal{L}(\mathcal{A}^-)$. Sabemos que existen $q_1 \in I^-$ y $q_2 \in F^-$ tal que $(q_1, v) \vdash_{\mathcal{A}^-}^* (q_2, \epsilon)$.

Por construcción de I^- y F^- , sabemos que existen $u, w \in \Sigma^*$ tal que $\hat{\delta}(q_0, u) = q_1$ y que $\hat{\delta}(q_2, w) \in F$.

Como las transiciones de δ y Δ son iguales, sabemos que $\hat{\delta}(q_1, v) = q_2$.

Por lo tanto, $\hat{\delta}(q_0, uvw) \in F$, y concluimos que $v \in \text{Mitad}(L)$.

Queda entonces demostrado que $\text{Mitad}(L)$ es un lenguaje regular.

La distribución de puntaje para esta solución es la siguiente:

- **(0.5 puntos)** Por identificar que existe un autómata cuyo lenguaje es L
- **(0.5 puntos)** Por definir los estados de $\mathcal{A}^-$ correctamente
- **(1 punto)** Por definir Δ^- correctamente
- Para I^- y F^- :
 - **[2 puntos]** Si se definen ambos correctamente
 - **[1 punto]** Si se definen, pero hay errores en su definición
 - **[0 puntos]** Si no se definen explícitamente
- **(2 puntos)** Por demostrar que los lenguajes son iguales (**1 punto** por cada dirección)

Solución 2: Saltarse u y w con copias del autómata que no leen letras

Se define el ϵ -NFA $\mathcal{A}^s = (Q^s, \Sigma, \Delta^s, I^s, F^s)$ a partir de $\mathcal{A}$, donde

- $Q^s = \{[q, i] \mid q \in Q, i \in \{1, 2, 3\}\}$ (Hacer tres copias de Q y enumerarlas)
- $\Delta_1^s = \{([q, 1], \epsilon, [\delta(q, a), 1]) \mid q \in Q, a \in \Sigma\}$ (Transiciones sin lectura en la primera copia)
- $\Delta_2^s = \{([q, 2], a, [\delta(q, a), 2]) \mid q \in Q, a \in \Sigma\}$ (Transiciones iguales que δ en la segunda copia)
- $\Delta_3^s = \{([q, 3], \epsilon, [\delta(q, a), 3]) \mid q \in Q, a \in \Sigma\}$ (Transiciones sin lectura en la tercera copia)
- $\Delta_{\rightarrow}^s = \{([q, i], \epsilon, [q, i+1]) \mid q \in Q, i \in \{1, 2\}\}$ (Pasar de una copia a la siguiente)
- $\Delta^s = \Delta_1^s \cup \Delta_2^s \cup \Delta_3^s \cup \Delta_{\rightarrow}^s$
- $I^s = \{[q_0, 1]\}$ (Empezar en la primera copia)
- $F^s = \{[p, 3] \mid p \in F\}$ (Aceptar sólo en la tercera copia)

P.D: $\text{Mitad}(L) = \mathcal{L}(\mathcal{A}^s)$

($\subseteq$) Supongamos que $v \in \text{Mitad}(L)$. Sabemos que existen $u, w \in \Sigma^*$ tal que $u \cdot v \cdot w \in L$.

Por lo tanto, existen $q_1, q_2 \in Q$ y $p \in F$ tal que $\hat{\delta}(q_0, u) = q_1$, $\hat{\delta}(q_1, v) = q_2$ y $\hat{\delta}(q_2, w) = p$.

Por construcción de Δ^s , sabemos lo siguiente:

$$([q_0, 1], v) \underbrace{\vdash_{\mathcal{A}^s}^*}_{\text{Por } \Delta_1^s} ([q_1, 1], v) \underbrace{\vdash_{\mathcal{A}^s}^*}_{\text{Por } \Delta_{\rightarrow}^s} ([q_1, 2], v) \underbrace{\vdash_{\mathcal{A}^s}^*}_{\text{Por } \Delta_2^s} ([q_2, 2], \epsilon) \underbrace{\vdash_{\mathcal{A}^s}^*}_{\text{Por } \Delta_{\rightarrow}^s} ([q_2, 3], \epsilon) \underbrace{\vdash_{\mathcal{A}^s}^*}_{\text{Por } \Delta_3^s} ([p, 3], \epsilon)$$

Por construcción, tenemos que $[q_0, 1] \in I^s$ y que $[p, 3] \in F^s$. Por lo tanto, $v \in \mathcal{L}(\mathcal{A}^s)$.

($\supseteq$) Supongamos que $v \in \mathcal{L}(\mathcal{A}^s)$. Notemos que la única forma de llegar a un estado final es usando las transiciones de $\Delta_{\rightarrow}^s$ para avanzar hasta la copia 3. Por lo tanto, existen $q_1, q_2 \in Q$ y $p \in F$ tal que

$$\underbrace{([q_0, 1], v) \vdash_{\mathcal{A}^s}^* ([q_1, 1], v) \vdash_{\mathcal{A}^s}^* ([q_1, 2], v)}_{(1)} \underbrace{\vdash_{\mathcal{A}^s}^* ([q_2, 2], \epsilon)}_{(2)} \underbrace{\vdash_{\mathcal{A}^s}^* ([q_2, 3], \epsilon) \vdash_{\mathcal{A}^s}^* ([p, 3], \epsilon)}_{(3)}$$

Por construcción de Δ_1^s y (1), sabemos que existe $u \in \Sigma^*$ tal que $\hat{\delta}(q_0, u) = q_1$.

Por construcción de Δ_2^s y (2), sabemos que $\hat{\delta}(q_1, v) = q_2$.

Por construcción de Δ_3^s y (3), sabemos que existe $w \in \Sigma^*$ tal que $\hat{\delta}(q_2, w) = p$.

Por lo tanto, tenemos que $\hat{\delta}(q_0, uvw) = p$.

Por construcción de F^s , $p \in F$. Por lo tanto, $v \in \text{Mitad}(L)$.

Queda entonces demostrado que $\text{Mitad}(L)$ es un lenguaje regular.

La distribución de puntaje para esta solución es la siguiente:

- **(0.5 puntos)** Por identificar que existe un autómata cuyo lenguaje es L
- **(0.5 puntos)** Por definir I^s y F^s correctamente
- **(1 punto)** Por definir los estados de $\mathcal{A}^s$ e incluir a Δ_2^s (o equivalente a δ) en la definición de Δ^s
- Para Δ_1^s , Δ_3^s y $\Delta_{\rightarrow}^s$:
 - **[2 puntos]** Si se definen correctamente
 - **[1 punto]** Si se definen, pero hay errores en su definición
 - **[0 puntos]** Si no se definen explícitamente
- **(2 puntos)** Por demostrar que los lenguajes son iguales (**1 punto** por cada dirección)

Pregunta 2

Considere el siguiente problema:

Problema:	MIRRORNFA
Input:	Un NFA $A = (Q, \Sigma, \Delta, I, F)$ y $w = a_1 \dots a_n \in \Sigma^*$.
Output:	TRUE si, y solo si, $\exists i \in \{1, \dots, n-1\}. a_1 \dots a_i \in \mathcal{L}(A) \wedge a_n a_{n-1} \dots a_{i+1} \in \mathcal{L}(A)$.

En otras palabras, la respuesta es TRUE si la palabra w se divide en dos partes, donde la parte de la izquierda está en $\mathcal{L}(A)$ y la de la derecha invertida está en $\mathcal{L}(A)$. Entregue un algoritmo que resuelva MIRRORNFA en tiempo $\mathcal{O}(|A| \cdot |w|)$ y explique la correctitud de su algoritmo.

Solución

Lo que haremos es ejecutar el algoritmo de **On-The-Fly** leyendo la palabra de izquierda a derecha leyendo $a_1 a_2 \dots a_n$ recordando todas las posiciones donde llegamos a un estado final y luego haremos una pasada de derecha a izquierda leyendo $a_n a_{n-1} \dots a_1$ buscando si es que podemos llegar a algun estado final y la posición anterior fue marcada como final por la pasada anterior.

Algorithm 1: MIRRORNFA

Input: Un NFA $A = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \dots a_n$
Output: TRUE si y solo si $\exists i \in \{1, \dots, n-1\}$ tal que $a_1 \dots a_i \in \mathcal{L}(A)$ y $a_n a_{n-1} \dots a_{i+1} \in \mathcal{L}(A)$.

```

 $B[1 \dots n] \leftarrow \text{FALSE}$ 
 $S \leftarrow I$ 
for  $i = 1$  to  $n$  do
     $S_{\text{old}} \leftarrow S$ 
     $S \leftarrow \emptyset$ 
    foreach  $p \in S_{\text{old}}$  do
         $S \leftarrow S \cup \{q \mid (p, a_i, q) \in \Delta\}$ 
    foreach  $p \in S$  do
        if  $p \in F$  then
             $B[i] \leftarrow \text{TRUE}$ 
 $S \leftarrow I$ 
for  $i = n$  to  $2$  do
     $S_{\text{old}} \leftarrow S$ 
     $S \leftarrow \emptyset$ 
    foreach  $p \in S_{\text{old}}$  do
         $S \leftarrow S \cup \{q \mid (p, a_i, q) \in \Delta\}$ 
    foreach  $p \in S$  do
        if  $p \in F$  and  $B[i-1] = \text{TRUE}$  then
            return TRUE
return FALSE

```

La demostración de correctitud es consecuencia de que sabemos que el algoritmo **On-The-Fly** es correcto. Por lo tanto, luego del primer for (que es solo aplicar el algoritmo clasico) sabemos que $B[i] = \text{True}$ si y solo si $a_1 a_2 \dots a_i \in \mathcal{L}(A)$. Luego para la pasada en reversa, tenemos que cuando la condicion de termino se cumple, $p \in F$ si y solo si $a_n a_{n-1} \dots a_i \in \mathcal{L}(A)$ y como $B[i-1] = \text{True}$ entonces $a_1 a_2 \dots a_{i-1} \in \mathcal{L}(A)$ cumpliendo asi lo pedido.

La complejidad es $\mathcal{O}(|A| \cdot |w|)$ ya que ejecutamos **On-The-Fly** 2 veces.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 puntos)** Por utilizar una estructura de datos que recuerde la informacion de la primera pasada

- **(1 puntos)** Por usar **On-The-Fly** de izquierda a derecha
- **(1 puntos)** Por actualizar correctamente la estructura de datos con la condicion de termino de la primera pasada
- **(1 puntos)** Por usar **On-The-Fly** de derecha a izquierda
- **(1 puntos)** Por la condición de termino correcta
- **(1 puntos)** Por la explicacion de la correctitud.

Es importante mencionar de que en caso de que el algoritmo entregado no cumpla la complejidad pedida, la respuesta no se considerará correcta. Por lo que el puntaje asignado queda a discrecion del ayudante corrector.

Pregunta 3

Pregunta 3.1

Una posible solución para esta pregunta es la siguiente construcción de la gramática:

$$\begin{aligned} S &\rightarrow X_{ad} \\ X_{ad} &\rightarrow aX_{ad}d \mid X_{ac} \mid X_{bd} \\ X_{ac} &\rightarrow aX_{ac}c \mid X_{bc} \\ X_{bd} &\rightarrow bX_{bd}d \mid X_{bc} \\ X_{bc} &\rightarrow bX_{bc}c \mid \epsilon \end{aligned}$$

Explicación de la gramática: Desde X_{ad} agrego x a's y d's, luego si paso a X_{ac} agrego y a's y c's, o si paso a X_{bd} agrego y b's y d's, finalmente en la producción X_{bc} agrego z b's y c's. En general tengo que,

$$\begin{aligned} \#a + \#b &= \#c + \#d \\ x + y + z &= x + y + z \end{aligned}$$

cumpliéndose la condición $i + j = p + q$. Además, podemos observar que el orden en el que se aplican las producciones preservan el orden de la palabra, tal que genera $a^i b^j c^p d^q$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (0.75 punto) Por la producción X_{ad}
- (0.75 punto) Por la producción X_{ac}
- (0.75 punto) Por la producción X_{bd}
- (0.75 punto) Por la producción X_{bc}

Pregunta 3.2

Solución a. Podemos crear un apilador alternativo a partir de la gramática anterior. La construcción sería la siguiente:

$$\mathcal{D} = (Q, \Sigma, \Delta, q_0, \{q_f\})$$

donde,

- $Q = \{q_0, q_f, A, B, C, D, S, X_{ad}, X_{ac}, X_{bd}, X_{bc}\}$
- $\Delta = (q_0, \epsilon, Sq_f) \cup \Delta_{prod} \cup \Delta_e$

En este caso Δ_{prod} son las transiciones que vienen de las producciones de la gramática, tal que:

$$\begin{aligned} \Delta_{prod} = & (S, \epsilon, X_{ad}) \cup \\ & (X_{ad}, \epsilon, AX_{ad}D) \cup (X_{ad}, \epsilon, X_{ac}) \cup (X_{ad}, \epsilon, X_{bd}) \cup (X_{ad}, \epsilon, X_{bc}) \cup \\ & (X_{ac}, \epsilon, AX_{ac}C) \cup (X_{ac}, \epsilon, X_{bc}) \cup \\ & (X_{bd}, \epsilon, BX_{bd}D) \cup (X_{bd}, \epsilon, X_{bc}) \cup \\ & (X_{bc}, \epsilon, BX_{bc}C) \cup (X_{bc}, \epsilon, \epsilon) \end{aligned}$$

y Δ_e son las transiciones que consumen la palabra y van eliminando los estados A, B, C y D del stack, tal que:

$$\Delta_e = (A, a, \epsilon) \cup (B, b, \epsilon) \cup (C, c, \epsilon) \cup (D, d, \epsilon)$$

Explicación: Utilizando las transiciones que vienen de la gramática el apilador construye de forma no determinista una palabra válida en el stack, luego las transiciones de eliminación revisan que la palabra recibida sea una palabra válida en la gramática comparándola con lo que está escrito en el stack.

Dado lo anterior, la distribución de puntaje es la siguiente:

- (0.5 puntos) Por definir los estados.
- (0.5 puntos) Por agregar la transición inicial (q_0, ϵ, Sq_f) .
- (1 punto) Por agregar las transiciones Δ_{prod} .
- (1 punto) Por agregar las transiciones Δ_e .

Solución b. Podemos crear un apilador alternativo que acepte la palabra, tal que:

$$\mathcal{D} = (Q, \Sigma, \Delta, q_a, \{q_d\})$$

donde,

- $Q = \{q_a, q_b, q_c, q_d, q\}$
- $\Delta = \begin{aligned} &(q_a, a, q_aq) \cup (q_a, \epsilon, q_b) \cup \\ &(q_b, b, q_bq) \cup (q_b, \epsilon, q_c) \cup \\ &(q_cq, c, q_c) \cup (q_c, \epsilon, q_d) \cup \\ &(q_dq, d, q_d) \end{aligned}$

Explicación: Primero se debe entender que los estados q_a, q_b, q_c y q_d se utilizan para marcar que sección de la palabra se está leyendo, de esta forma uno no puede 'devolverse' a una fase anterior, es decir, no se puede leer las letras en desorden. Luego, el estado q sirve como contador, por cada a y b añadimos una q y por cada c y d eliminamos una q , de forma que si $\#a + \#b = \#c + \#d$, entonces la cantidad de q añadidas y eliminadas es la misma, alcanzando el estado final q_d .

Dado lo anterior, la distribución de puntaje es la siguiente:

- (0.5 puntos) Por definir los estados.
- (0.75 puntos) Por agregar las transiciones con a y b que suman q .
- (0.75 puntos) Por agregar las transiciones con c y d que restan q .
- (1 punto) Por agregar las ϵ -transiciones que cambian de estado.

Pregunta 4

Para cada uno de los siguientes lenguajes determine si es (1) regular, (2) libre de contexto y no regular, o (3) no libre de contexto. Demuestre su respuesta.

1. $L_1 = \{a^n b^m c^\ell \mid n < m \wedge m < \ell\}$

2. $L_2 = \{a^n b^m c^\ell \mid n < m \vee m < \ell\}$

Hint: L_1 y L_2 difieren en su categorización entre (1), (2) y (3).

Solución

4.1) $L_1 = \{a^n b^m c^\ell \mid n < m < \ell\}$

L_1 No es libre de contexto. Usaremos el lema de bombeo para lenguajes libres de contexto. Para todo $N > 0$, tomemos la palabra:

$$z = a^N b^{N+1} c^{N+2} \in L_1,$$

pues claramente:

$$N < N + 1 < N + 2.$$

Por el lema, existe una descomposición:

$$z = u v w x y$$

tal que:

$$|vwx| \leq N, \quad |vx| > 0,$$

Como $|vwx| \leq N$, el segmento vwx puede ubicarse únicamente en los siguientes casos:

$$\begin{cases} (1) & vwx \in \mathcal{L}(a^*), \\ (2) & vwx \in \mathcal{L}(a^*b^+), \\ (3) & vwx \in \mathcal{L}(b^+c^*), \\ (4) & vwx \in \mathcal{L}(c^*), \end{cases}$$

Mostraremos que en todos los casos existe un i tal que

$$uv^iwx^iy \notin L_1.$$

Caso (1): $vwx \in \mathcal{L}(a^*)$

Si bombeamos con $i = 2$:

$$\#a(uv^2wx^2y) > N + 1 > N.$$

Pero $\#b = N + 1$, y tenemos que:

$$\#a \geq \#b,$$

lo cual viola $n < m$. Luego $uv^2wx^2y \notin L_1$.

Caso (2): $vwx \in \mathcal{L}(a^*b^+)$

Bombear con $i = 2$ aumenta las b :

$$\#b(uv^2wx^2y) > N + 2 > N + 1.$$

Lo que genera:

$$\#b \geq \#c,$$

contradiendo $m < \ell$. Por tanto $uv^2wx^2y \notin L_1$.

Caso (3): $vw \in \mathcal{L}(b^+c^*)$

Bombeando hacia abajo, con $i = 0$:

$$\#b(uv^0wx^0y) \leq N$$

Teniendo entonces que:

$$\#b \leq \#a,$$

violando $n < m$. Entonces $uv^0wx^0y \notin L_1$.

Caso (4): $vw \in \mathcal{L}(c^*)$

Bombeando hacia abajo, con $i = 0$:

$$\#c(uvwxy) \leq N.$$

$$\#c(uv^0wx^0y) \leq N - 1 < N + 1.$$

Teniendo entonces que:

$$\#c \leq \#b,$$

violando $m < \ell$. Entonces $uv^0wx^0y \notin L_1$.

Conclusión.

L_1 no es libre de contexto.

- (0.5 ptos) Por indicar que el lenguaje no es libre de contexto.
- (0.5 ptos) Por elegir una palabra correcta.
- (2 ptos) Por casos, 0.5 por caso.

4.2) $L_2 = \{ a^n b^m c^\ell \mid n < m \vee m < \ell \}$.

Es libre de contexto (a) y no regular (b).

(a) Diseñar una gramática libre de contexto que genere L_2 .

Queremos generar palabras donde:

$$n < m \quad \text{y} \quad m < \ell.$$

Sea entonces la gramática

$$\mathcal{G} = (\{S, X_a, X_b, X_c, X_{ab}, X_{bc}\}, \{a, b, c\}, P, S)$$

La idea es generar pasos de “crecimiento escalonado” entre los bloques, usando no terminales auxiliares. Tenemos entonces las siguientes producciones:

$$S \rightarrow X_{ab} X_c \mid X_a X_{bc}$$

$$X_a \rightarrow a X_a \mid \varepsilon$$

$$X_{ab} \rightarrow a X_{ab} b \mid X_b b$$

$$X_b \rightarrow b X_b \mid \varepsilon$$

$$X_{bc} \rightarrow b X_{bc} c \mid X_c c$$

$$X_c \rightarrow c X_c \mid \varepsilon$$

Estas reglas permiten “transferir” control entre bloques de forma que X_b produzca más símbolos que X_a , y X_c más que X_b , asegurando $n < m < \ell$.

(b) Demostrar que L_2 no es regular.

Usaremos el lema de Bombeo para lenguajes regulares.

Para todo $N > 0$ consideremos la palabra

$$w = xyz \in L_2 \quad \text{con } x = \varepsilon, \quad y = a^N \quad \text{y} \quad z = b^{N+1}c$$

se descompone y tal que

$$y = uvw = a^{k_1} a^{k_2} a^{k_3} \quad \text{con } k_2 \neq 0 \quad \text{y} \quad k_1 + k_2 + k_3 = N$$

Bombeamos con $i = 2$,

$$xy^2z = a^{k_1+2k_2+k_3} b^{N+1}c$$

Pero esta palabra ya no satisface la condición $n < m$ porque:

$$\#a(xy^2z) > \#b(xy^2z)$$

Por lo que $a^{k_1+2k_2+k_3} b^{N+1}c \notin L_2$ y L_2 no es regular.

Conclusión:

L_2 es libre de contexto y no regular.

- **(0.5 ptos)** (a) Por indicar que el lenguaje es libre de contexto.

- **(0.5 ptos)** (a) Por hacer la disyunción inicial de casos en la gramática.
- **(0.5 ptos)** (a) Por mantener la desigualdad en las producciones.
- **(0.5 ptos)** (b) Por escoger una palabra correcta.
- **(0.5 ptos)** (b) Por escoger i correcto y uso correcto del lema.
- **(0.5 ptos)** (b) Por demostrar que la palabra bombeada no pertenece a L_2 .